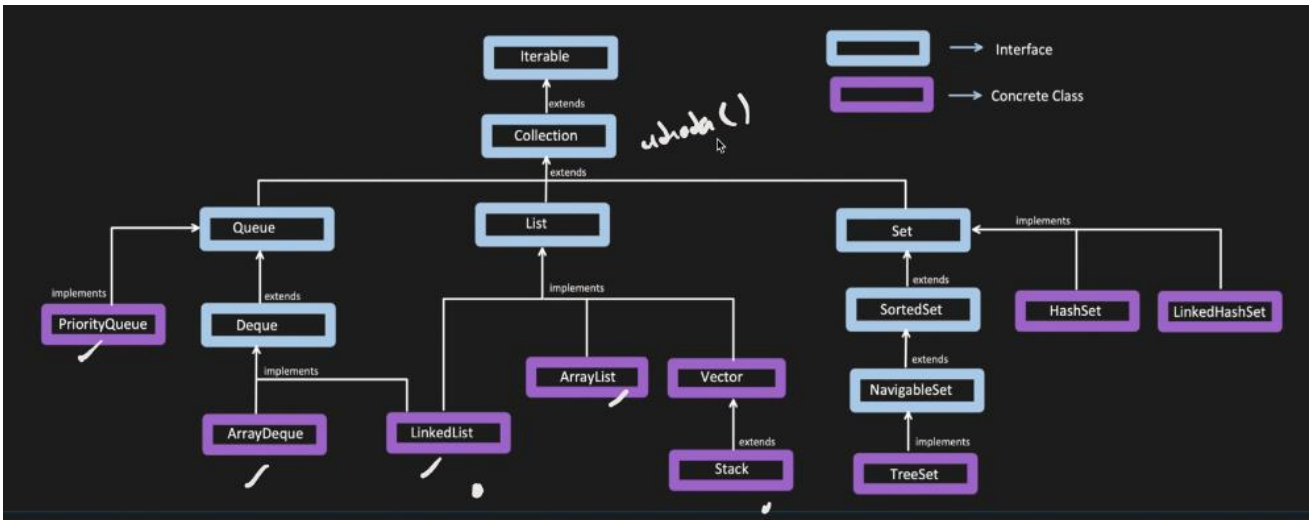
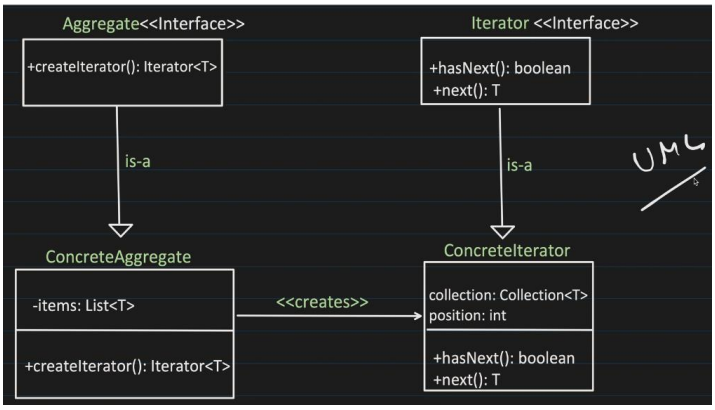
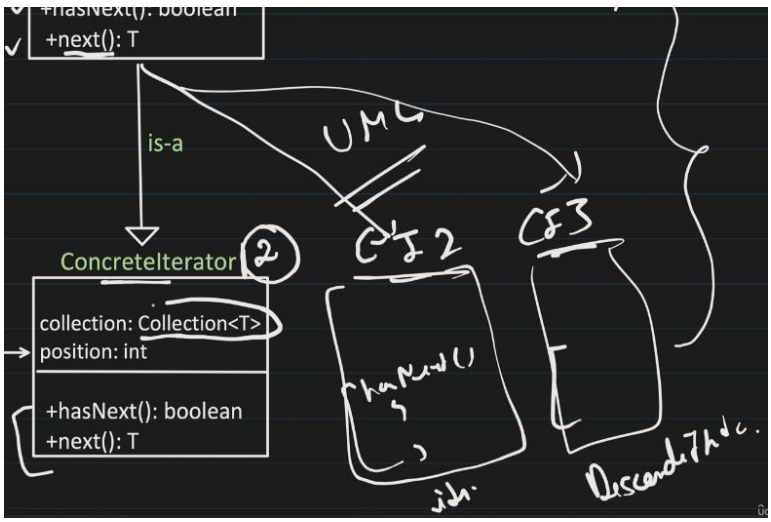
29. Iterator Design Pattern (Behavioral Design Pattern)

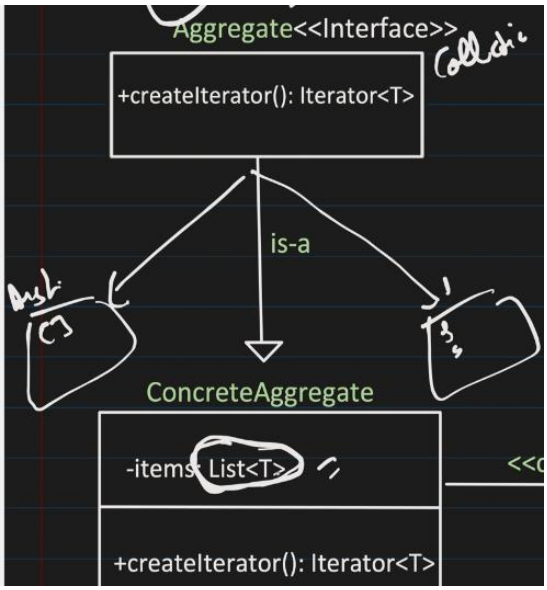


So, its a Behavioral Design pattern, that provides a way to access element of a Collection sequentially without exposing the underlying representation of the collection.

```
public class LinkedHashSetExample {

    public static void main(String args[]){

        Set<Integer> intSet = new LinkedHashSet<>();
        intSet.add(2);
        intSet.add(77);
        intSet.add(82);
        intSet.add(63);
        intSet.add(5);

        Iterator<Integer> iterable = intSet.iterator();
        while(iterable.hasNext()){
            int val = iterable.next();
            System.out.println(val);
        }
```







Eg : Book and Library

```java
public class Client {

    public static void main(String[] args) {
        List<Book> booksList = Arrays.asList(
                new Book( price: 100,  bookName: "Science"),
                new Book( price: 200,  bookName: "Maths"),
                new Book( price: 300,  bookName: "GK"),
                new Book( price: 400,  bookName: "Drawing")
        );

        Library lib = new Library(booksList);
        Iterator iterator = lib.createIterator();

        while (iterator.hasNext()) {
            Book book = (Book) iterator.next();
            System.out.println(book.getBookName());
        }
    }
}
```

```java
public class Book {

    private int price;
    private String bookName;

    Book(int price, String bookName){
        this.price = price;
        this.bookName = bookName;
    }
    public int getPrice() {
        return price;
    }

    public String getBookName() {
        return bookName;
    }
}
```

Lets see one Example: Books and Library

```java
public interface Aggregate {

    Iterator createIterator();
}
```

```java
public interface Iterato
    boolean hasNext();
    Object next();
}
```

is-a

```java
public class Library {
```

```java
public class BookIterator implements
```

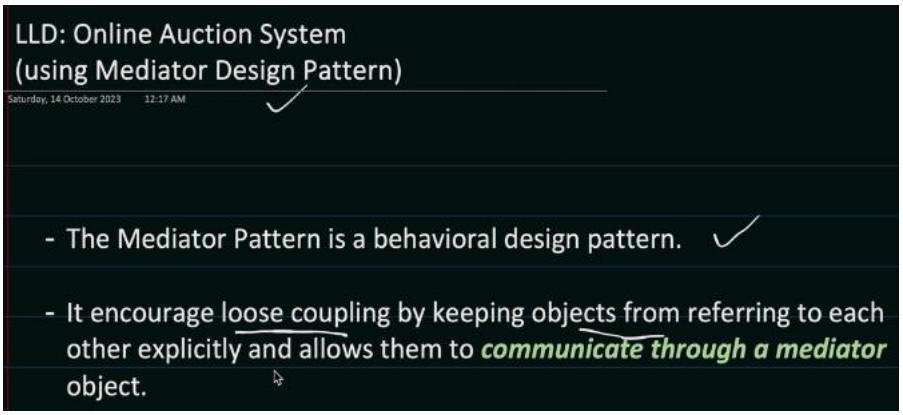
```java
public class Library {

    private List<Book> booksList;

    public Library(List<Book> booksList) {
        this.booksList = booksList;
    }

    public Iterator createIterator() {
        return new BookIterator(booksList);
    }
}
```

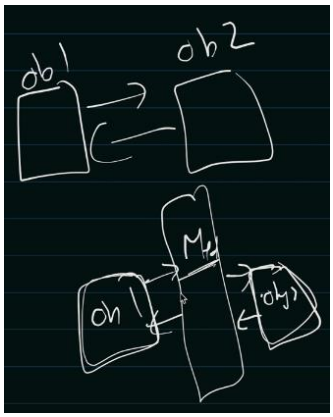
```
ksList) {
t;

() {
ooksList);
```

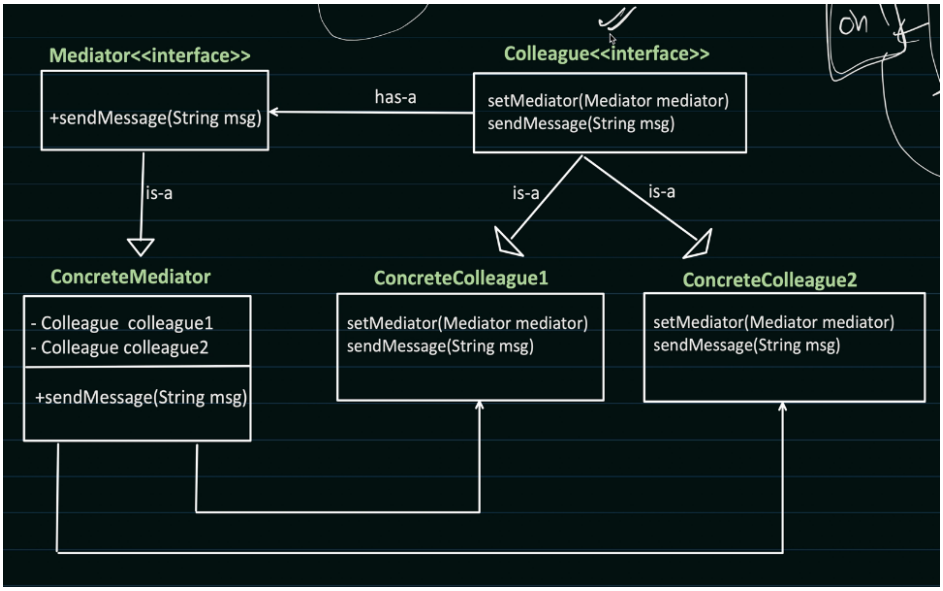
<<creates>>

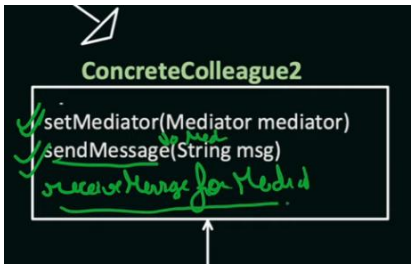
```java
public class BookIterator implements Iterator {
    private List<Book> books;
    private int index = 0;

    public BookIterator(List<Book> books) {
        this.books = books;
    }

    @Override
    public boolean hasNext() {
        return index < books.size();
    }

    @Override
    public Object next() {
        if (this.hasNext()) {
            return books.get(index++);
        }
```

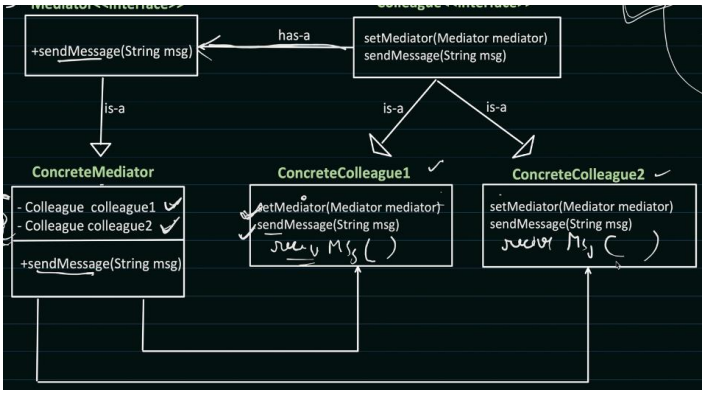
30. Mediator Design Pattern (Behavioral Design Pattern)

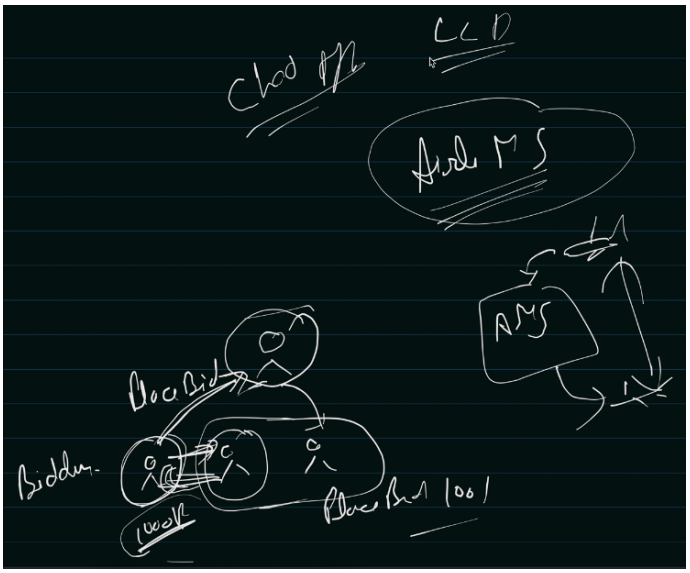








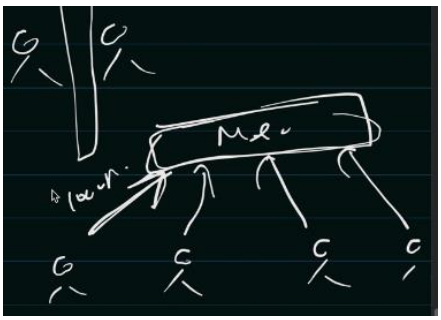




UML : Lets use, Online Auction System Example to understand the UML

<< Interface >>

```java
public interface Colleague {

    void placeBid(int bidAmount);
    void receiveBidNotification(int bidAmount);
    String getName();
}
```



in-a

```java
public class Bidder implements Colleague {

    String name;
    AuctionMediator auctionMediator;

    Bidder(String name, AuctionMediator auctionMediator) {
        this.name = name;
        this.auctionMediator = auctionMediator;
        auctionMediator.addBidder(this);
    }

    @Override
    public void placeBid(int bidAmount) {
        auctionMediator.placeBid( bidder: this, bidAmount);
    }

    @Override
    public void receiveBidNotification(int bidAmount) {
        System.out.println("Bidder: " + name + " got the notification that someone has put bid of : " + bidAmount);
    }

    @Override
    public String getName(){
        return name;
    }
}
```

<< Interface >>

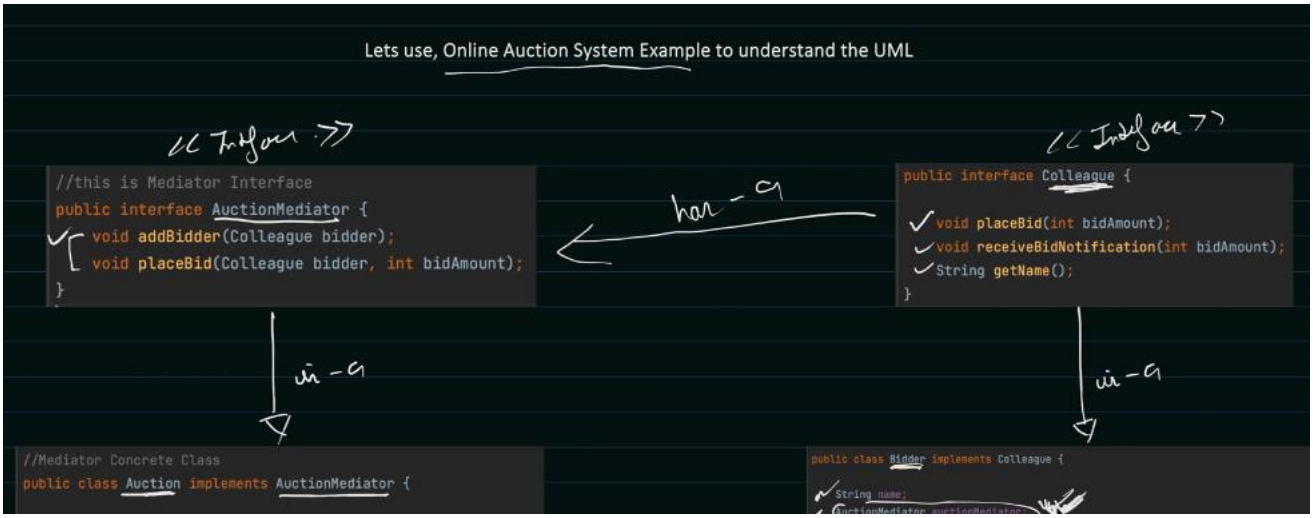
```java
//this is Mediator Interface
public interface AuctionMediator {
    void addBidder(Colleague bidder);
    void placeBid(Colleague bidder, int bidAmount);
}
```

in-a

```java
//Mediator Concrete Class
public class Auction implements AuctionMediator {
```

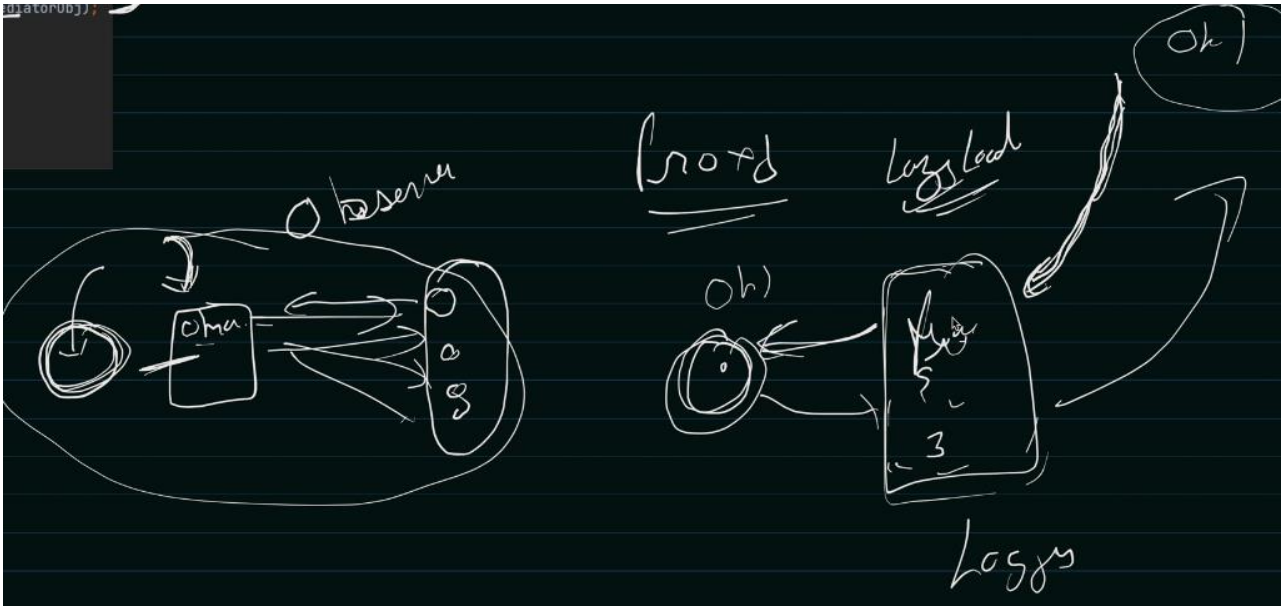
in-a

```java
//Mediator Concrete Class
public class Auction implements AuctionMediator {

    List<Colleague> colleagues = new ArrayList<>();

    @Override
    public void addBidder(Colleague bidder) {
        colleagues.add(bidder);
    }

    @Override
    public void placeBid(Colleague bidder, int bidAmount) {

        for(Colleague colleague : colleagues){
            if(!colleague.getName().equals(bidder.getName())){
                colleague.receiveBidNotification(bidAmount);
            }
        }
    }
```




```
public class Main {

    public static void main(String args[]){

        AuctionMediator auctionMediatorObj = new Auction();
        Colleague bidder1 = new Bidder( name: "A", auctionMediatorObj);
        Colleague bidder2 = new Bidder( name: "B", auctionMediatorObj);

        bidder1.placeBid( bidAmount: 2000);
        bidder2.placeBid( bidAmount: 3000);
        bidder1.placeBid( bidAmount: 3001);
    }
}
```

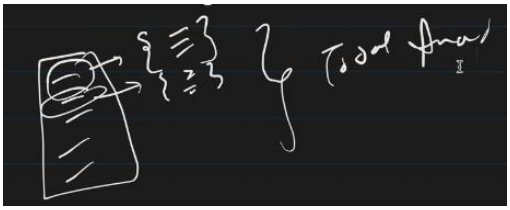
https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/behavioralpatterns/mediator?ref_type=heads

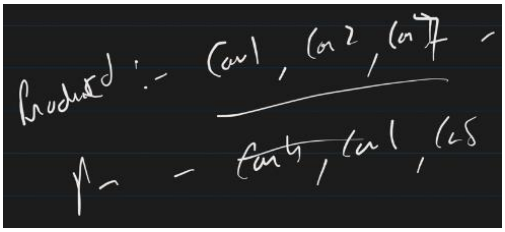


31. LLD of Apply Coupons on Shopping Cart Products

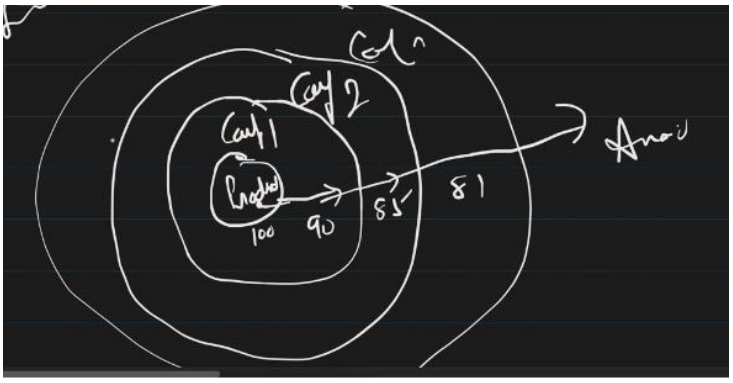
"Given Shopping cart with products and coupons and calculate the net price after applying coupons on products. Coupons can be of different types with certain conditions.
1. N% off that is 10% off for all the individual
2.P% off on next item
3.D% off on Nth item of Type T.
Sequentially wants to apply all the coupons on the cart and get the Total amount."

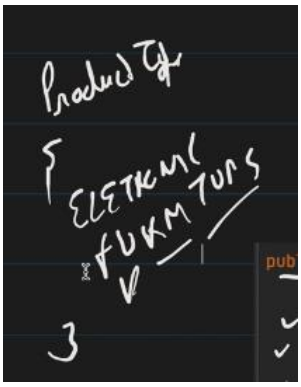
for each product there is a possible way that multiple sequence of coupons can be applied based upon the conditions
So, each product might be eligible for more than one coupon.





decorator pattern -
where you have an original product - without touching that original product, we can add certain capabilities to it.
we apply coupon 1 to it - let's say it was ₹100 after coupon one, it would be let's say ₹90.
We are again decorating it with a second coupon. After this it would be 85.
We are again decorating it with more coupons. After this it is 81 and ultimately the final amount.
Similarly for each product, we can decorate it, and ultimately, whatever the amount we will get,
We will sum it up and return the total amount.





```java
public class Main {

    public static void main(String[] args) {

        Product item1 = new Item1( name: "FAN",  originalPrice: 1000, ProductType.ELECTRONICS_GOODS);
        Product item2 = new Item2( name: "SOFA",  originalPrice: 2000, ProductType.FURNITURE_GOODS);

        ShoppingCart cart = new ShoppingCart();
        cart.addToCart(item1);
        cart.addToCart(item2);

        System.out.println("total price after discount:" + cart.getTotalPrice());
    }
}
```

```java
public abstract class Product{

    String name;
    double originalPrice;
    ProductType type;

    Product(){}

    Product(String name, double price, ProductType type){
        this.name = name;
        this.originalPrice = price;
        this.type = type;
    }

    public abstract double getPrice();

    public ProductType getType() {
        return type;
    }
}
```

```java
public class ShoppingCart {

    List<Product> productList;

    public ShoppingCart(){
        productList = new ArrayList<>();
    }

    public void addToCart(Product product){
        Product productWithEligibleDiscount =
                new TypeCouponDecorator(
                new PercentageCouponDecorator(product,  percentage: 10),  percentage: 3, product.getType());

        productList.add(productWithEligibleDiscount);
    }

    public int getTotalPrice(){

        int totalPrice = 0;
        for(Product product : productList){
            totalPrice += product.getPrice();
        }
        return totalPrice;
    }
}
```

```java
public class Item1 extends Product{

    Item1(String name, double originalPrice, ProductType type){
        super(name, originalPrice, type);
    }

    @Override
    public double getPrice() {
        return originalPrice;
    }
}
```

```java
public class Item2  extends Product{

    Item2(String name, double originalPrice, ProductType type){
        super(name, originalPrice, type);
    }

    @Override
    public double getPrice() {
        return originalPrice;
    }
}
```

```
ype){
```

has-a

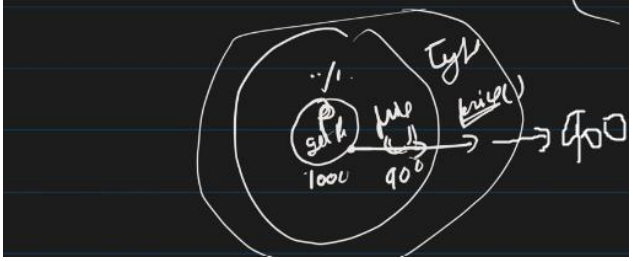
```java
public class PercentageCouponDecorator extends CouponDecorator {

    Product product;
    int discountPercentage;
    PercentageCouponDecorator(Product product, int percentage){
        this.product = product;
        this.discountPercentage = percentage;
    }

    @Override
    public double getPrice() {
        double price = product.getPrice();
        return price - (price * discountPercentage)/100;
    }
}
```

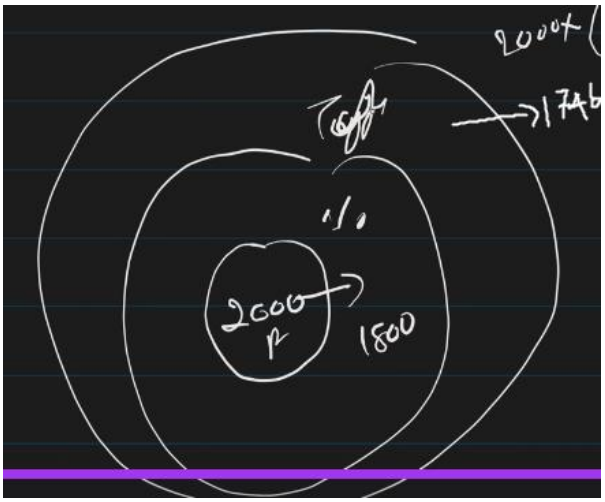
```java
public class TypeCouponDecorator extends CouponDecorator {

    Product product;
    int discountPercentage;
    ProductType type;
    static List<ProductType> eligibleTypes = new ArrayList<>();
    static {
        eligibleTypes.add(ProductType.FURNITURE_GOODS);
        eligibleTypes.add(ProductType.DECORATIVE_GOODS);
    }

    TypeCouponDecorator(Product product, int percentage, ProductType type){
        this.product = product;
        this.discountPercentage = percentage;
        this.type = type;
    }


    @Override
    public double getPrice() {
        double price = product.getPrice();
        if(eligibleTypes.contains(type)) {
            return price - (price * discountPercentage) / 100;
        }
```

```java
public double getPrice() {
    double price = product.getPrice();
    return price - (price * discountPercentage)/100;
}
```

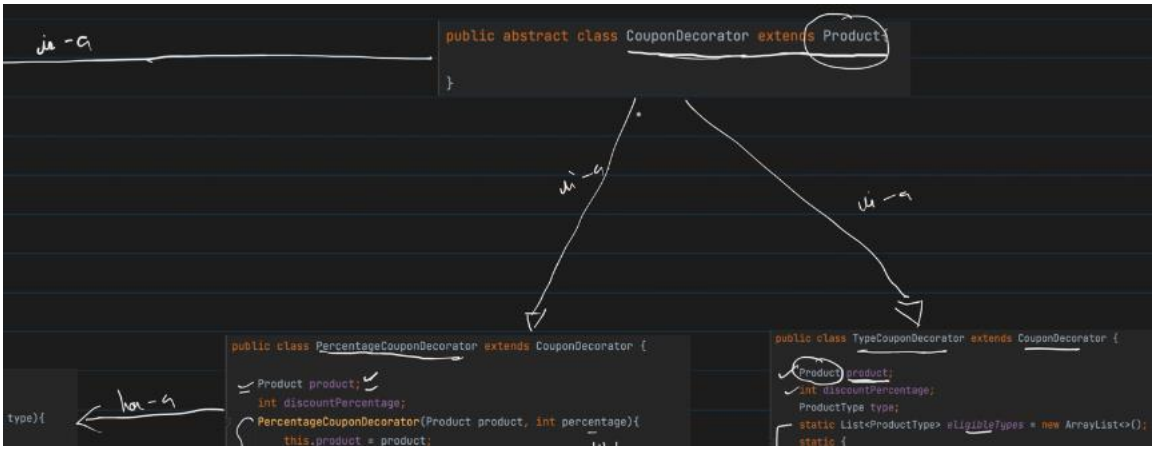
1000 - (100 * 10) / 100

=> 900





```
/Library/Java/JavaVirtualMachines/JDK-
total price after discount:2646

Process finished with exit code 0
```



32. Visitor Design Pattern (Behavioral Design Pattern)

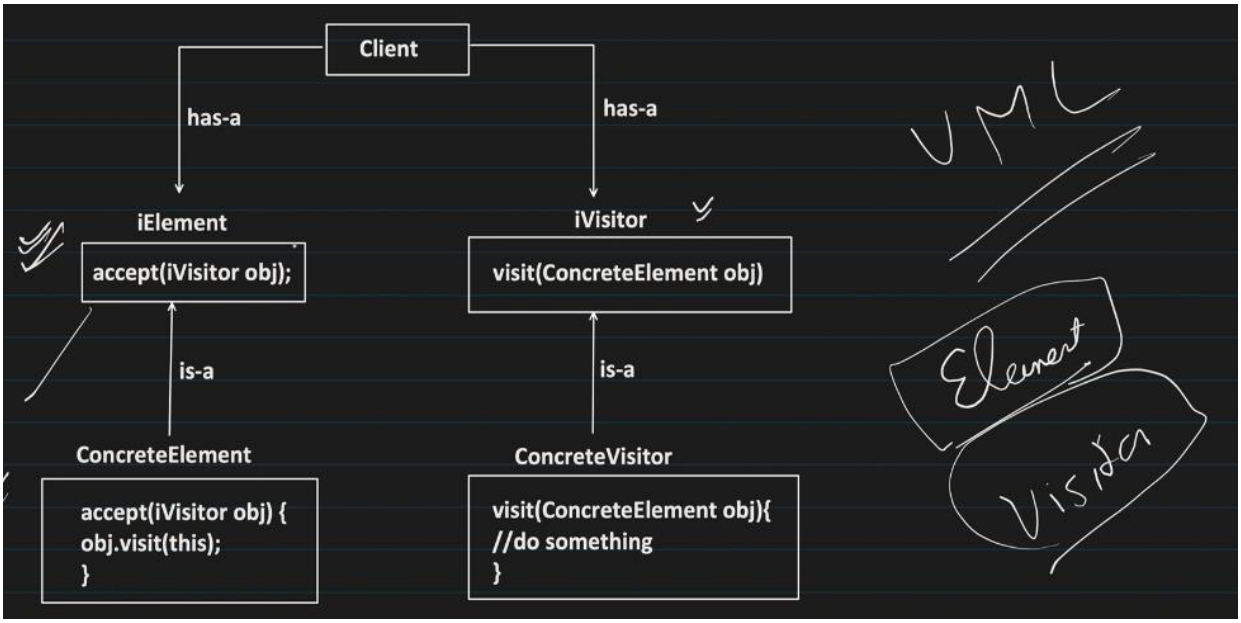
LLD: Hotel Booking System (using Visitor Design Pattern)

Sunday, 29 October 2023 7:38 PM

Whats the problem with the below class?

```
public class HotelRoom {

    public void getRoomPrice(){
        //price computation logic
    }

    public void initiateRoomMaintenance(){
        //start room maintenance
    }

    public void reserveRoom(){
        //perform operation to reserve the room
    }

    //many more operations can come over the time
}
```

- It is a **Behavioral design pattern**
- That allows you to add new operations to existing classes without changing their structure.
- It achieves this by separating the algorithm from the objects on which it operates.
- It does **Double Dispatch** to achieve this.
  (Double Dispatch means, method which need to be invoked decided by the caller object and the object passed in the argument.)



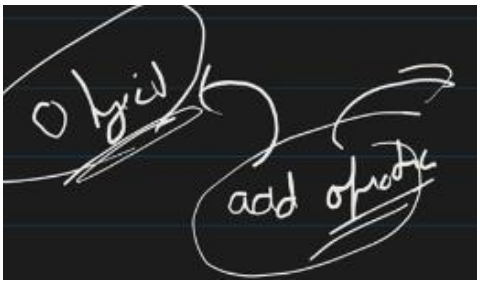
```java
public interface RoomElement {

    public void accept(RoomVisitor visitor);
}
```

is-a

```java
public class SingleRoom implements RoomElement{

    public int roomPrice = 0;

    @Override
    public void accept(RoomVisitor visitor) {
        visitor.visit( singleRoomObj: this);
    }
}
```

```java
public class DoubleRoom implements RoomElement{

    public int roomPrice = 0;

    @Override
    public void accept(RoomVisitor visitor) {
        visitor.visit( doubleRoomObj: this);
    }
}
```

```java
public class DeluxeRoom implements RoomElement{
```




```java
public class DeluxeRoom implements RoomElement{

    public int roomPrice = 0;

    @Override
    public void accept(RoomVisitor visitor) {
        visitor.visit( deluxeRoomObj: this);
    }
}
```

```java
public interface RoomVisitor {

    public void visit(SingleRoom singleRoomObj);
    public void visit(DoubleRoom doubleRoomObj);
    public void visit(DeluxeRoom deluxeRoomObj);
}
```

is-a

```java
public class RoomPricingVisitor implements RoomVisitor{

    @Override
```

```java
public class RoomMaintenanceVisitor
```

1

```java
public class RoomPricingVisitor implements RoomVisitor{

    @Override
    public void visit(SingleRoom singleRoomObj) {
        System.out.println("Pricing computation logic of SingleRoom");
        singleRoomObj.roomPrice = 1000;
    }

    @Override
    public void visit(DoubleRoom doubleRoomObj) {
        System.out.println("Pricing computation logic of DoubleRoom");
        doubleRoomObj.roomPrice = 2000;
    }

    @Override
    public void visit(DeluxeRoom deluxeRoomObj) {
        System.out.println("Pricing computation logic of DeluxeRoom");
        deluxeRoomObj.roomPrice = 5000;
    }
}
```

2

```java
public class RoomMaintenanceVisitor implements RoomVisitor{

    @Override
    public void visit(SingleRoom singleRoomObj) {
        System.out.println("Performing maintenance of SingleRoom");
    }

    @Override
    public void visit(DoubleRoom doubleRoomObj) {
        System.out.println("Performing maintenance of DoubleRoom");
    }

    @Override
    public void visit(DeluxeRoom deluxeRoomObj) {
        System.out.println("Performing maintenance of DeluxeRoom");
    }
}
```

3

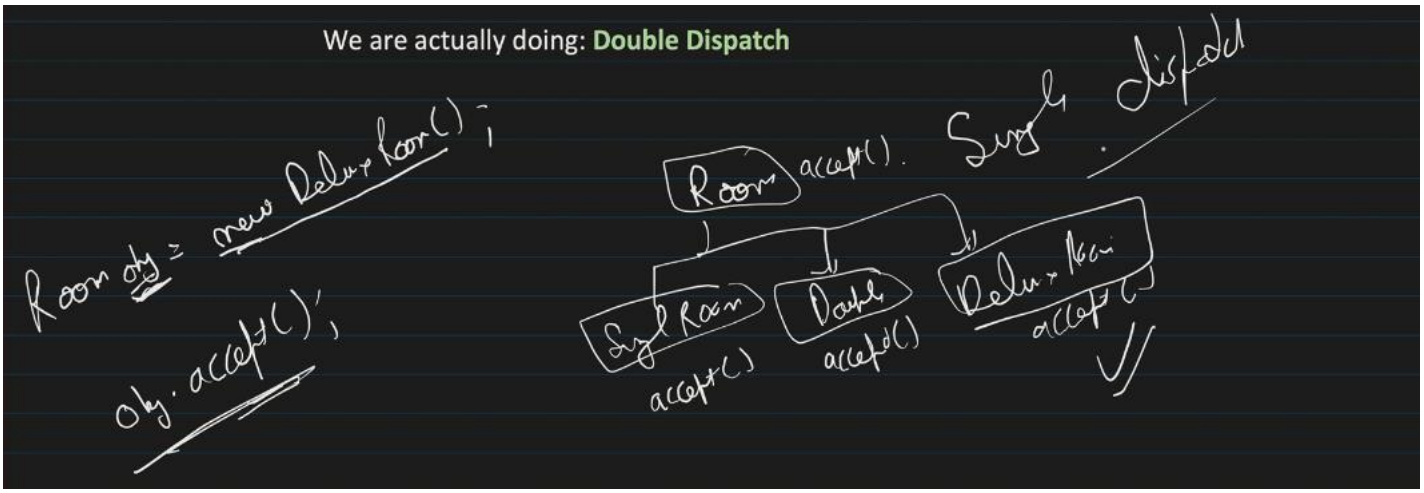
Room Visitor

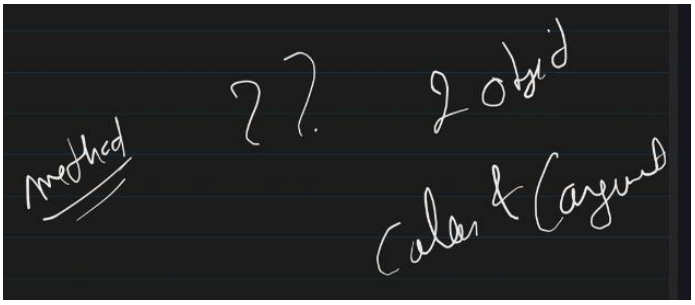
```java
public class Client {

    public static void main(String args[]){

        RoomElement singleRoomObj = new SingleRoom();
        RoomElement doubleRoomObj = new DoubleRoom();
        RoomElement deluxeRoomObj = new DeluxeRoom();

        //performing an operation on the  objects
        RoomVisitor pricingVisitorObj = new RoomPricingVisitor();
        singleRoomObj.accept(pricingVisitorObj);
        System.out.println(((SingleRoom)singleRoomObj).roomPrice);

        doubleRoomObj.accept(pricingVisitorObj);
        System.out.println(((DoubleRoom)doubleRoomObj).roomPrice);

        deluxeRoomObj.accept(pricingVisitorObj);
        System.out.println(((DeluxeRoom)deluxeRoomObj).roomPrice);

        //performing another operation on the  objects
        RoomVisitor maintenanceVisitorObj = new RoomMaintenanceVisitor();
        singleRoomObj.accept(maintenanceVisitorObj);

        doubleRoomObj.accept(maintenanceVisitorObj);

        deluxeRoomObj.accept(maintenanceVisitorObj);
    }
}
```

e Dispatch to achieve this.

tch means, method which need to be invoked decided by the caller object and the object passed in the argument.)

We are actually doing: **Double Dispatch**






```java
public class SingleRoom implements RoomElement{

    public int roomPrice = 0;

    @Override
    public void accept(RoomVisitor visitor) {
        visitor.visit( singleRoomObj: this);
    }
}
```

1

do 2nd disp.

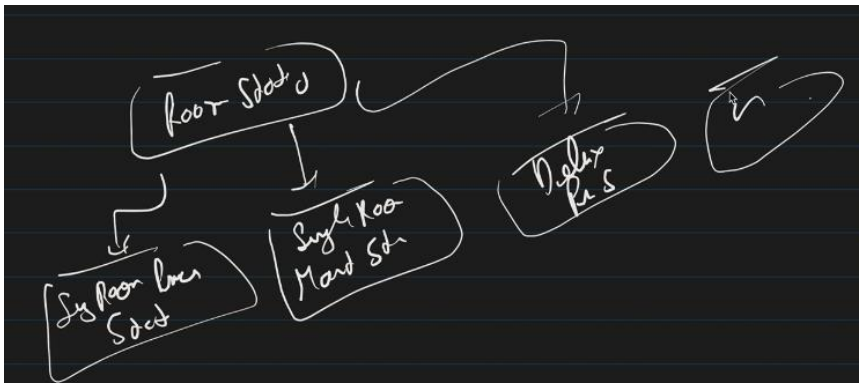
Single Dispatch

```java
public class
```

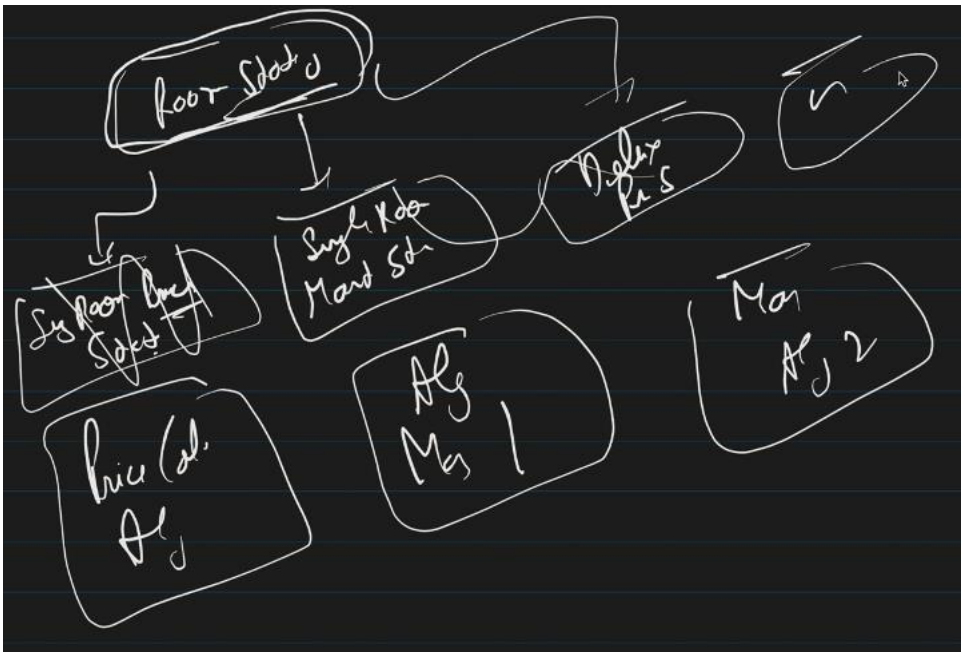
```java
public void accept(RoomVisitor visitor) {
    visitor.visit( singleRoomObj: this);
}
```

1 2

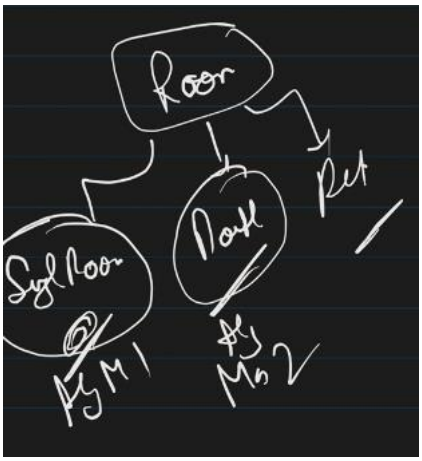
Visitor vs Strategy :



I can dynamically runtime pass which strategy and I can use it? No, this is wrong.
See these are all operations (Pricing, Maintenance etc. ...). These are not algorithms; these are operations



Strategy pattern is mostly for separating out the algorithm.
So strategy pattern separates out the algorithm.
Algorithms are independent of these elements like single room double room.
So algorithm can be like price calculation algo something, maintenance algo 1, maintenance algo 2 something.
Now the rooms (single / double) can decide which algorithm to use?



There is a difference between algorithms and operation.

You can't have a strategy for each operation.
strategy is for separating out the algorithm which is independent of this object.

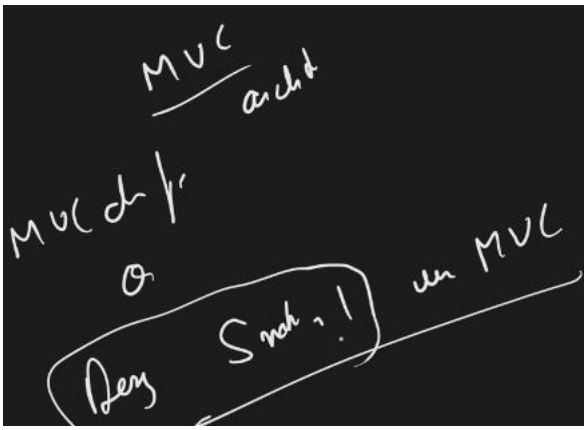
visitor pattern - operations are specifically for each of these elements – single/double/deluxe

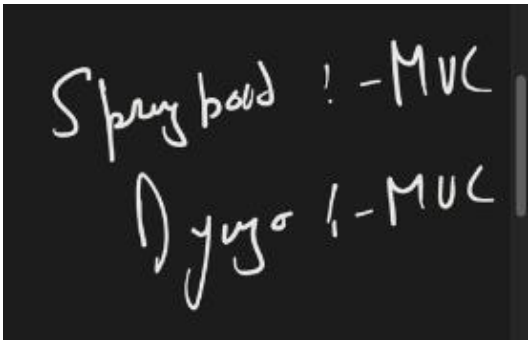
https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/behavioralpatterns/visitor?ref_type=heads

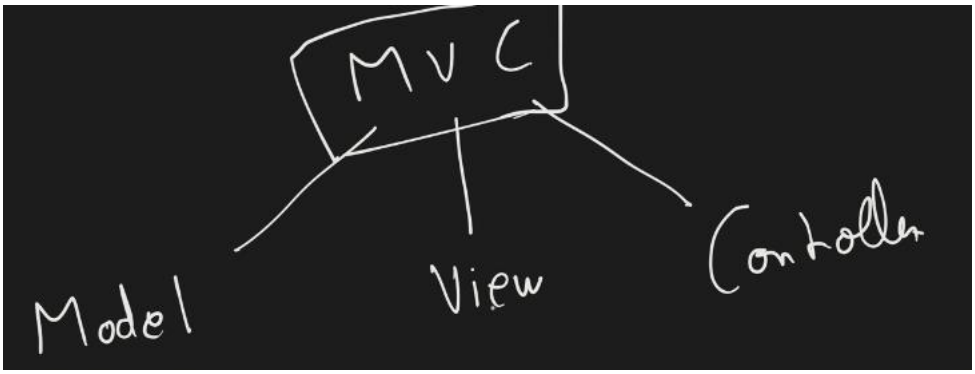
Visitor Pattern: Allows adding new operations to existing classes without modifying them

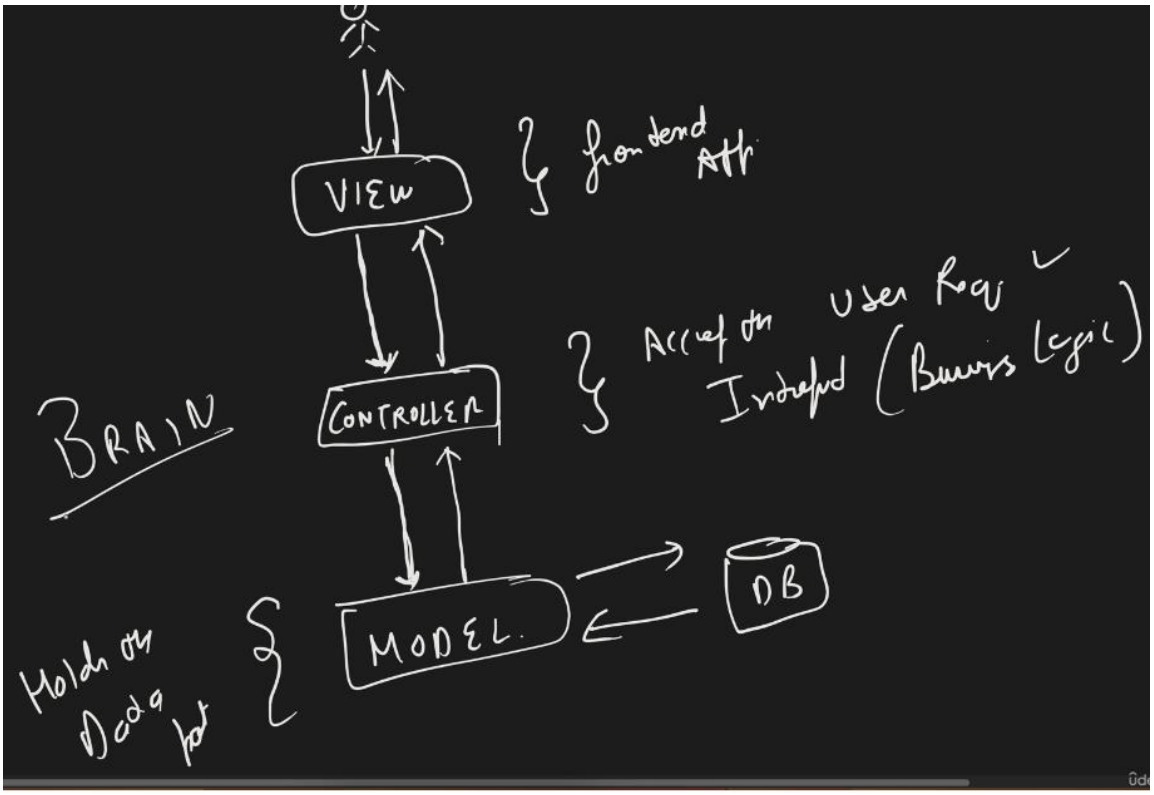
Visitor Pattern encourages OPEN & CLOSED Principle

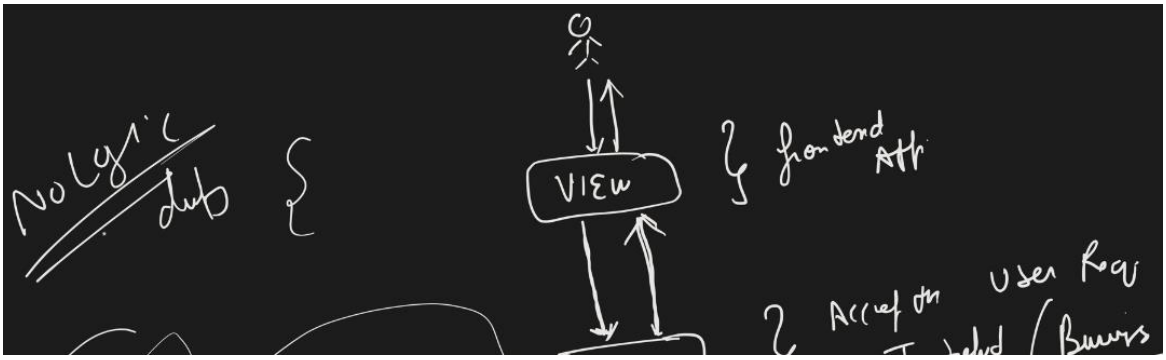
33. MVC Design Pattern

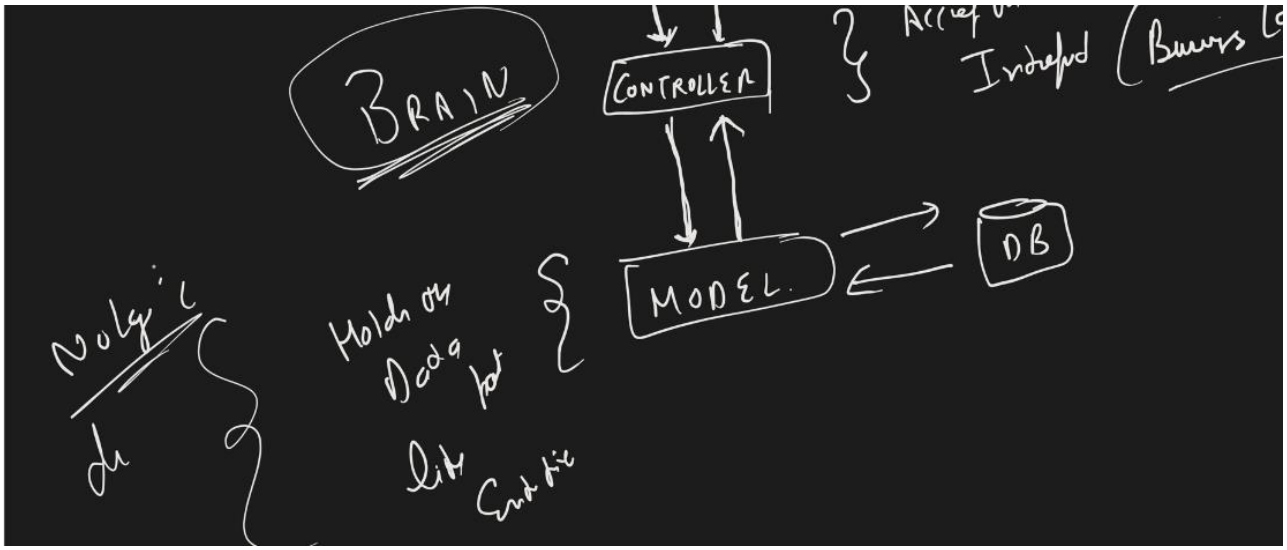


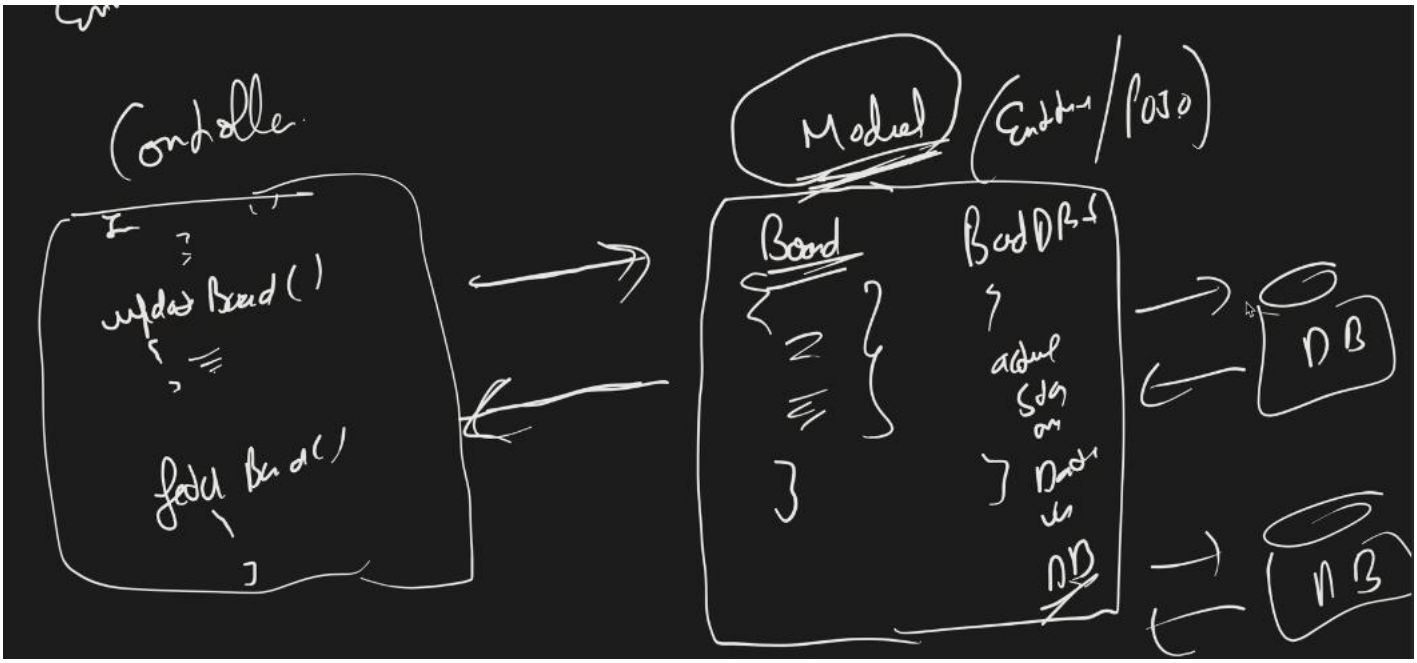


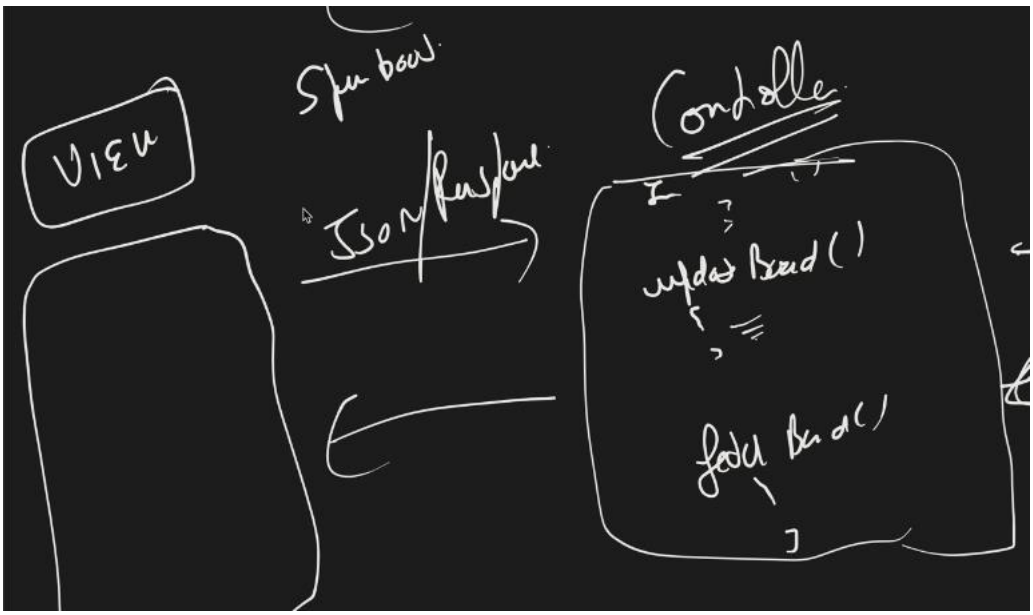


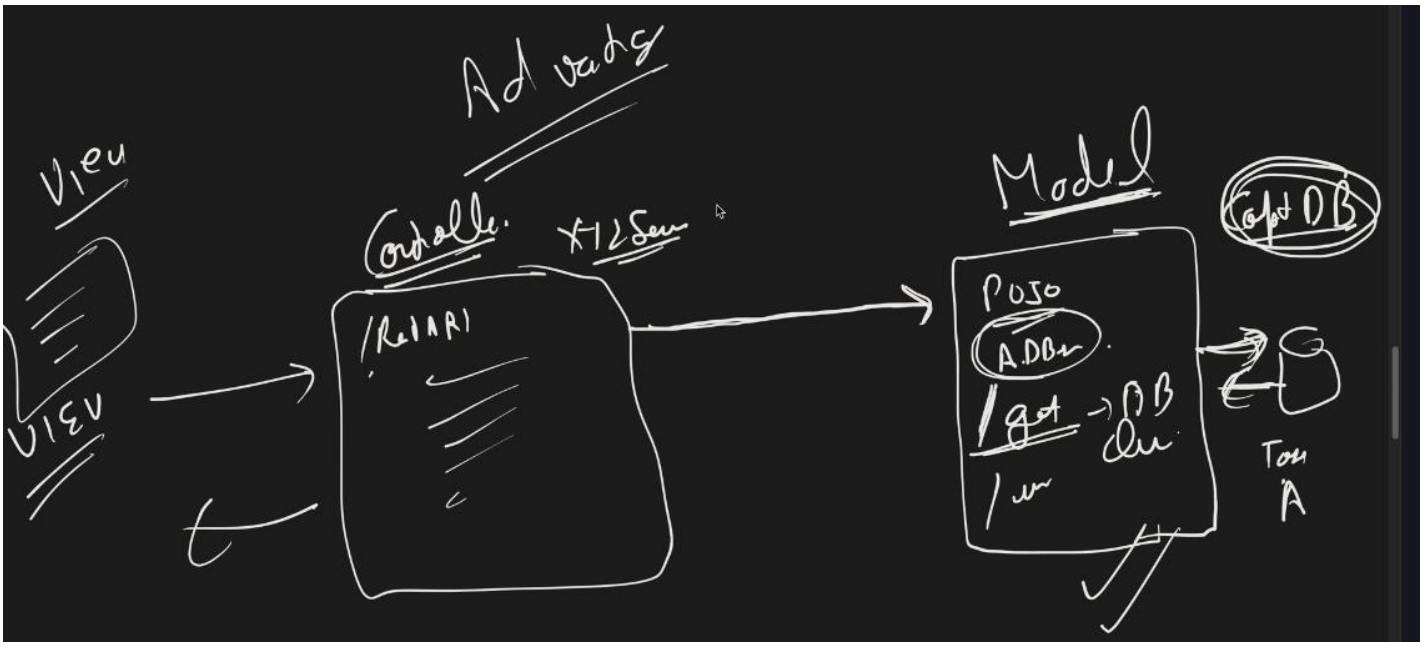


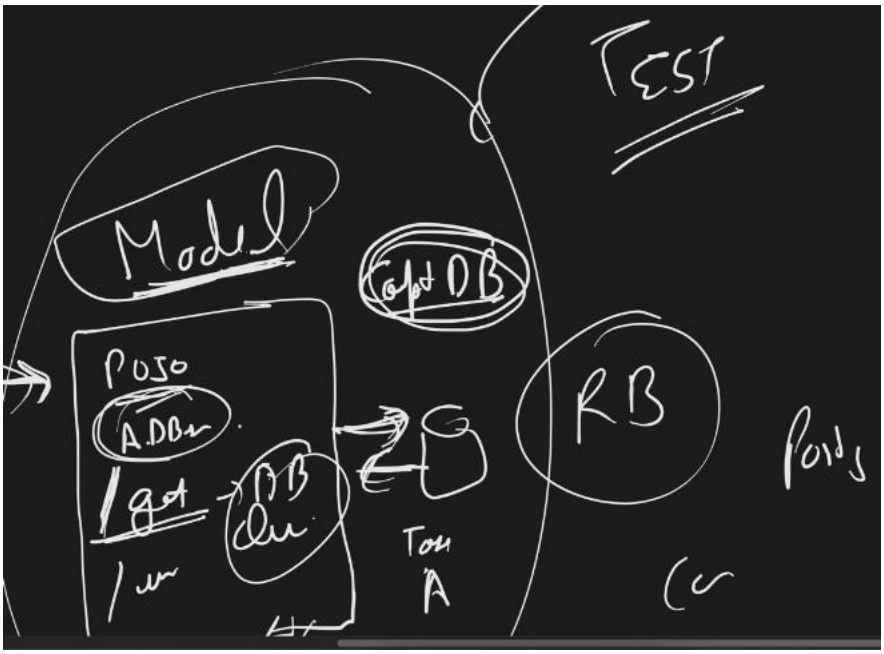



BRAIN
CONTROLLER
MODEL
DB


Controller
updateBrand()
fetchBrand()
Model
(Entity/POJO)
Brand
BrandDBS
actual
DB

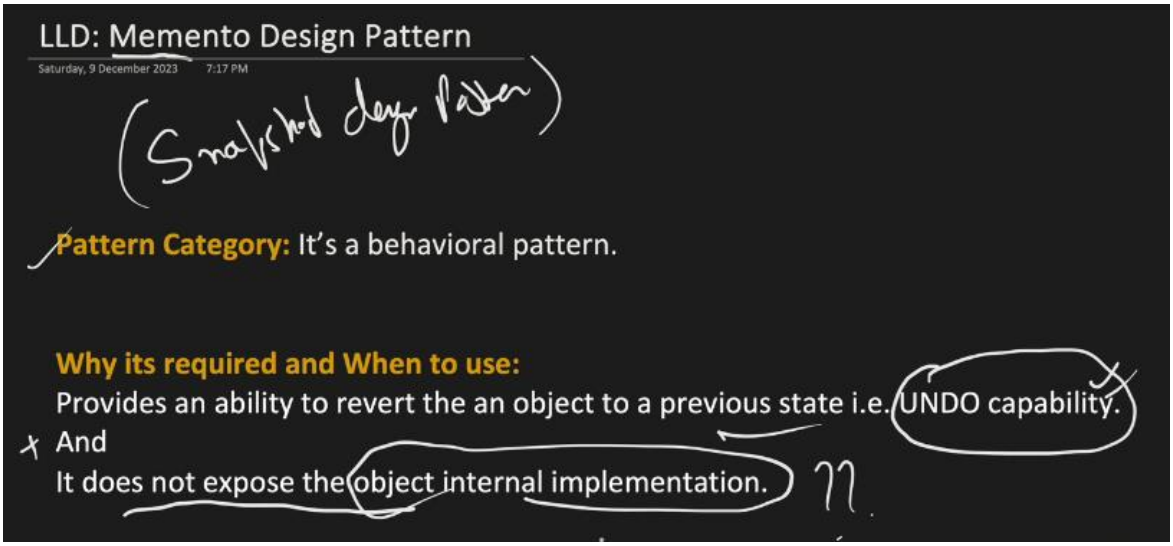

VIEW
Spring boot
Controller
updateBrand()
fetchBrand()



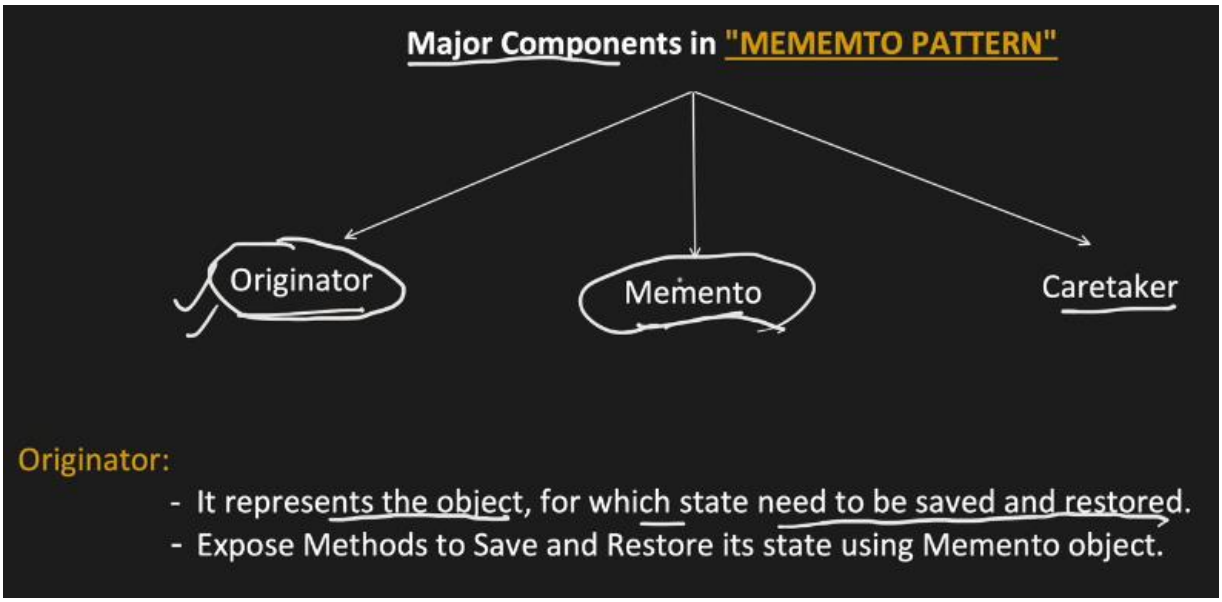


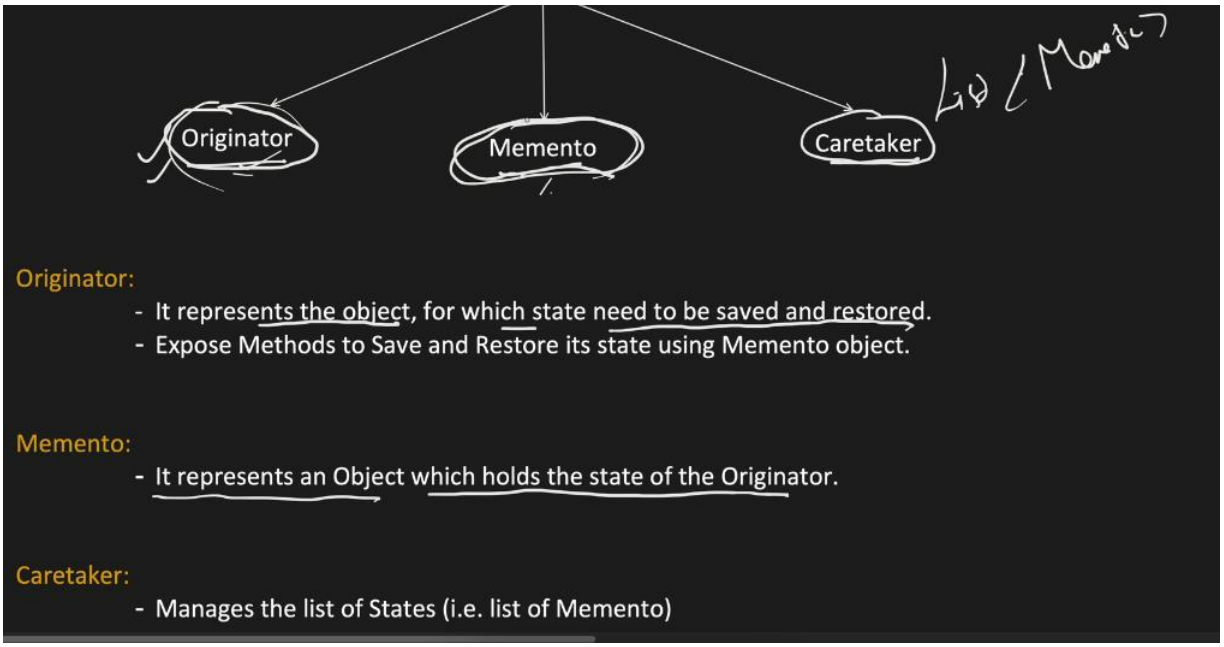
Easy to Maint
Enl to test
De Capt
Loose Capt

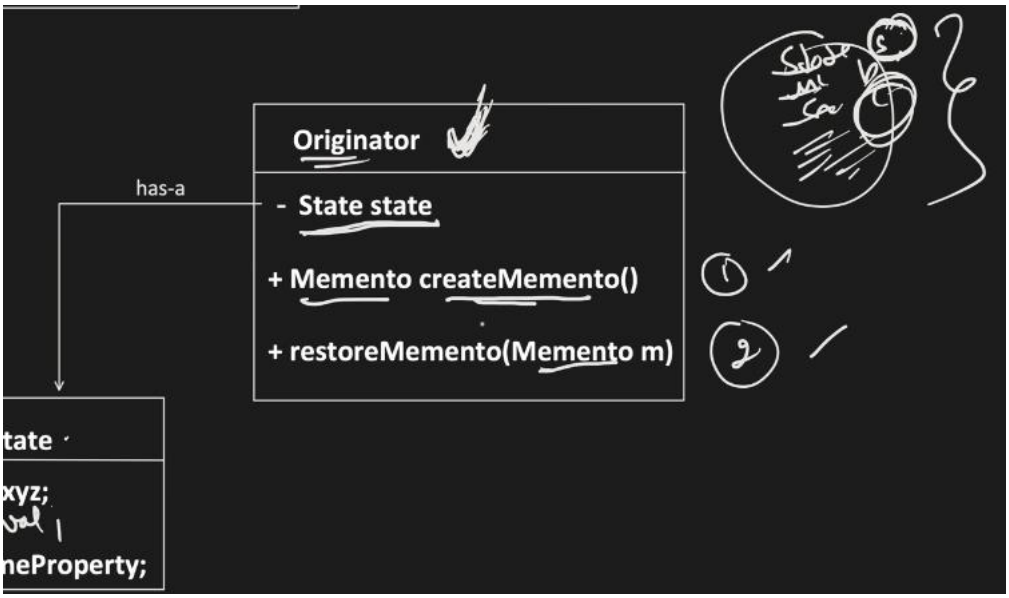
34. Memento Design Pattern (Behavioral Design Pattern)

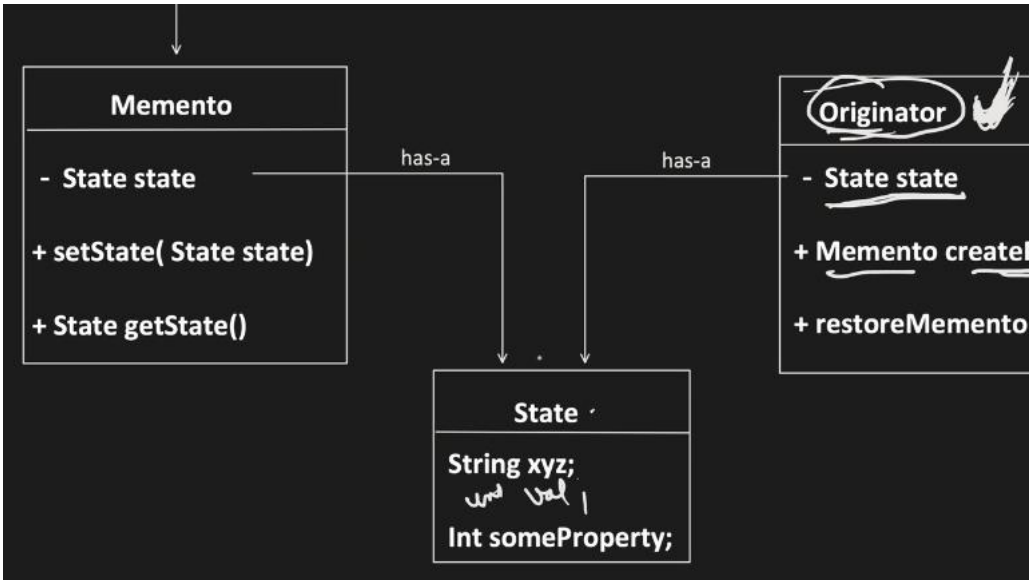


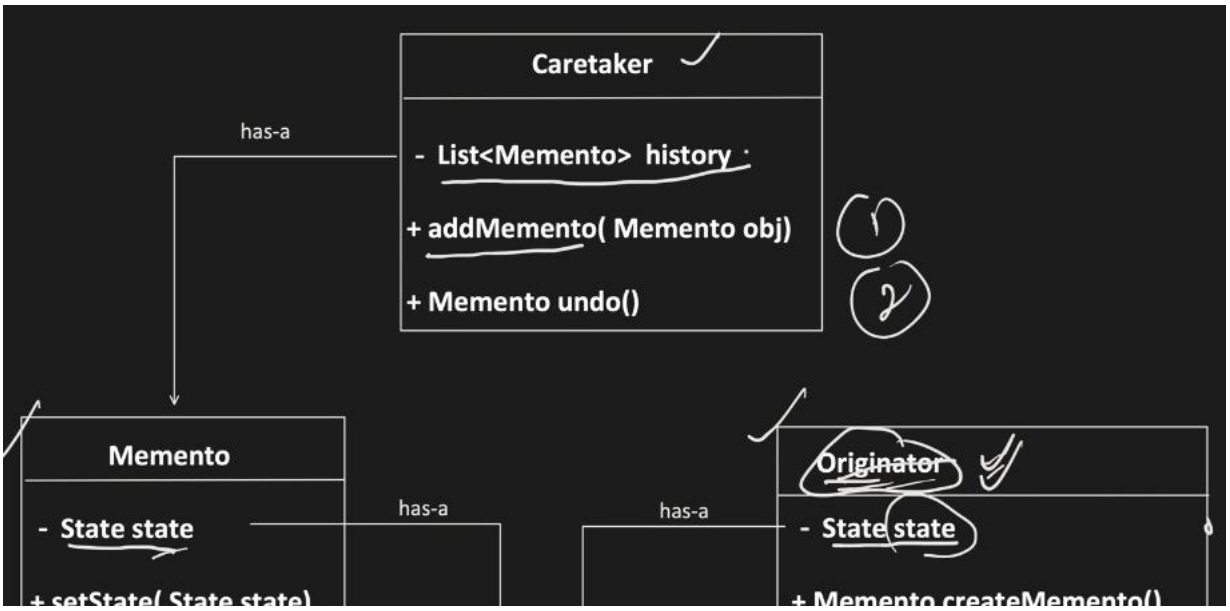
https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/behavioralpatterns/memento?ref_type=heads






Originator
- State state
+ Memento createMemento()
+ restoreMemento(Memento m)
has-a
tate
xyz;
neProperty;


Memento
- State state
+ setState(State state)
+ State getState()
has-a
has-a
Originator
- State state
+ Memento create
+ restoreMemento
State
String xyz;
Int someProperty;


Caretaker
- List<Memento> history
+ addMemento(Memento obj)
+ Memento undo()
has-a
Memento
- State state
has-a
has-a
Originator
- State state

originator

```java
//Originator
public class ConfigurationOriginator {

    int height;
    int width;

     ConfigurationOriginator(int height, int width){
        this.height = height;
        this.width = width;
    }

    public void setHeight(int height) {
        this.height = height;
    }

    public void setWidth(int width) {
        this.width = width;
    }

    public ConfigurationMemento createMemento(){
```

```java
    public ConfigurationMemento createMemento(){
        return new ConfigurationMemento(this.height, this.width);
    }

    public void restoreMemento(ConfigurationMemento mementoToBeRestored){
        this.height = mementoToBeRestored.height;
        this.width = mementoToBeRestored.width;
    }
}
```

memento

```java
//Memento
public class ConfigurationMemento {

    int height;
    int width;

    public ConfigurationMemento(int height, int width){
        this.height = height;
        this.width = width;
    }
```

```java
    public ConfigurationMemento(int height, int width){
        this.height = height;
        this.width = width;
    }

    public int getHeight() {
        return height;
    }

    public int getWidth() {
        return width;
    }
}
```

Caretaker

```java
//Caretaker
public class ConfigurationCareTaker {

    List<ConfigurationMemento> history = new ArrayList<>();

    public void addMemento(ConfigurationMemento memento) {
        history.add(memento);
    }

    public ConfigurationMemento undo() {
        if (!history.isEmpty()) {
            int lastMementoIndex = history.size() - 1;
            //get the last memento from the list
            ConfigurationMemento lastMemento = history.get(lastMementoIndex);
            //remove the last memento from the list now
            history.remove(lastMementoIndex);
            return lastMemento;
        }
        return null;
    }
```

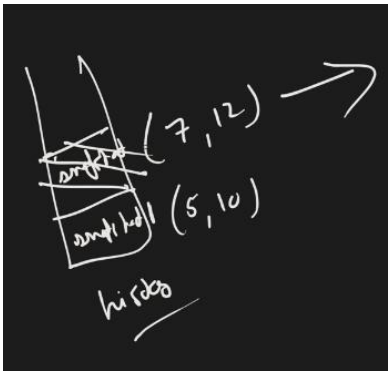
Client

```java
public class Client {

    public static void main(String args[]){

        ConfigurationCareTaker careTakerObject = new ConfigurationCareTaker();
        //initiate State of the originator
        ConfigurationOriginator originatorObject = new ConfigurationOriginator( height: 5, width: 10);

        //save it
        ConfigurationMemento snapshot1 = originatorObject.createMemento();

        //add it to history
        careTakerObject.addMemento(snapshot1);

        //originator changing to new state
        originatorObject.setHeight(7);
        originatorObject.setWidth(12);

        //save it
        ConfigurationMemento snapshot2 = originatorObject.createMemento();

        //add it to history
        careTakerObject.addMemento(snapshot2);
```

```java
originatorObject.setHeight(7);
originatorObject.setWidth(12);

//save it
ConfigurationMemento snapshot2 = originatorObject.createMemento();

//add it to history
careTakerObject.addMemento(snapshot2);

//originator changing to new state
originatorObject.setHeight(9);
originatorObject.setWidth(14);

//UNDO
ConfigurationMemento restoredStateMementoObj = careTakerObject.undo();
originatorObject.restoreMemento(restoredStateMementoObj);

System.out.println("height: " + originatorObject.height + " width: " + originatorObject.width );
```

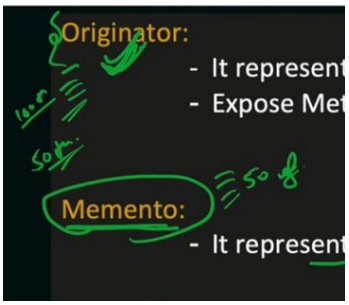
9, 14

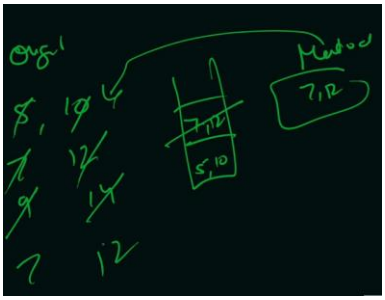
7, 12



Memory

Memento Pattern: Provides an ability to revert an object to a previous state i.e. UNDO capa
and it does not expose the object internal implementation.





35. Template Method Design Pattern (Behavioral Design Pattern)

LLD: Template Method Design Pattern

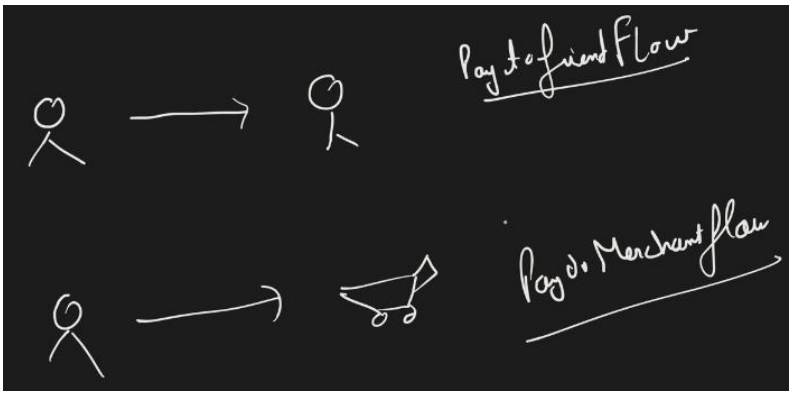
Thursday, 21 December 2023 2:26 PM

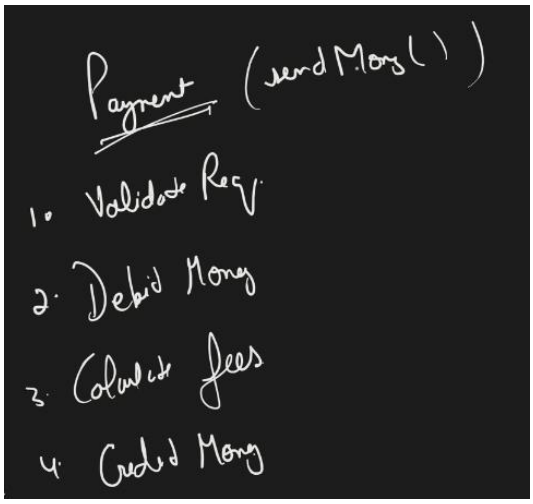
Pattern Category: It's a behavioral pattern.

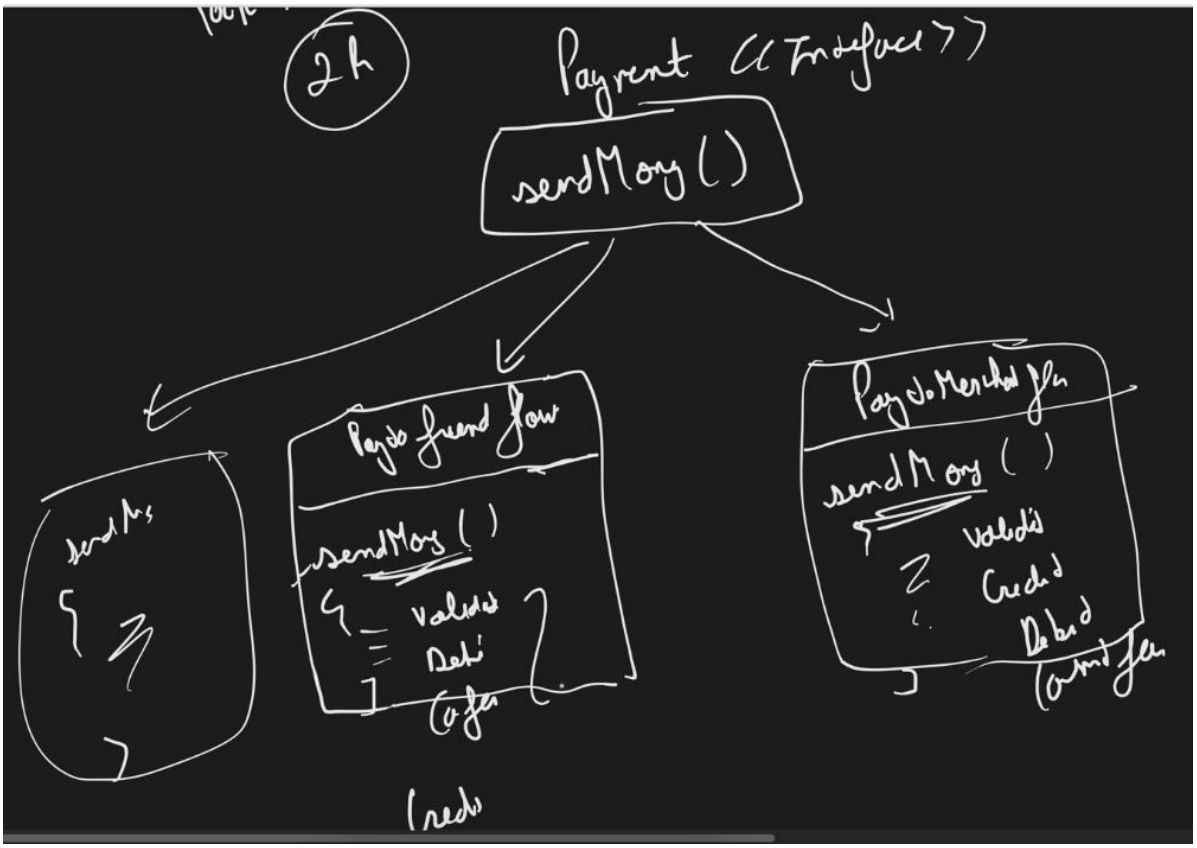
Why its required and When to use:

When you want all classes to follow the specific steps to process the task but also

Need to provide the flexibility that each class can have their own logic in that specific steps.





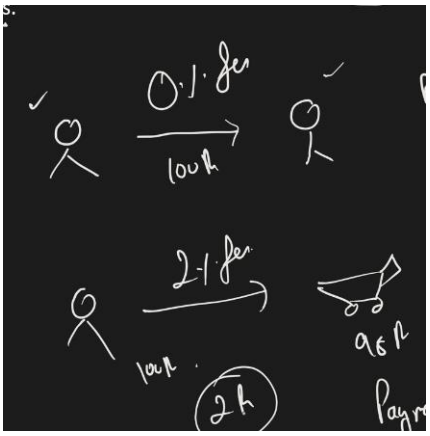


First:
I have a payment interface, has send money method.
one child is pay to friend, and another child would be Pay to merchant flow.
So, they will override Send money and they provide their own implementation.
For this send money we want all these classes to follow these specific steps validate, then debit, then calculate fees and then credit money.

our requirement is that we want all classes to follow the specific steps to process the task.
All the child classes of this payment, no matter if any future, any more flows comes.
If anybody is doing send money, they need to process a specific steps.

So, pay to friend can first validate and then debit and then compute fees and then credit.
But it is possible that Pay to merchant will validate, then credit and then debit and then compute fees. who will guarantee that they are following the same sequence? they have to follow order.

Second :
I also need to provide the flexibility that each class can have their own specific logic and specific steps.
So for validate request() might be pay to friend flow might be doing certain validation.
Pay to merchant might be doing certain differently.
For Calculate fees pay to friend do not charge any fees 0% fees.
So person is sending ₹100 - person is receiving ₹100.
But pay to merchant is charging 2% fees. If a person is buying ₹100 thing, merchant is getting only ₹98 to rupees.
Each flow can have different logic for these steps.



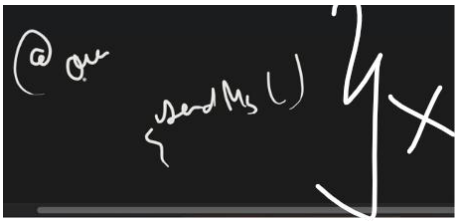
When all child classes, you want them to follow a specific step to process the task,
and also flexibility that each class can have their own logic in those specific steps - we have template pattern

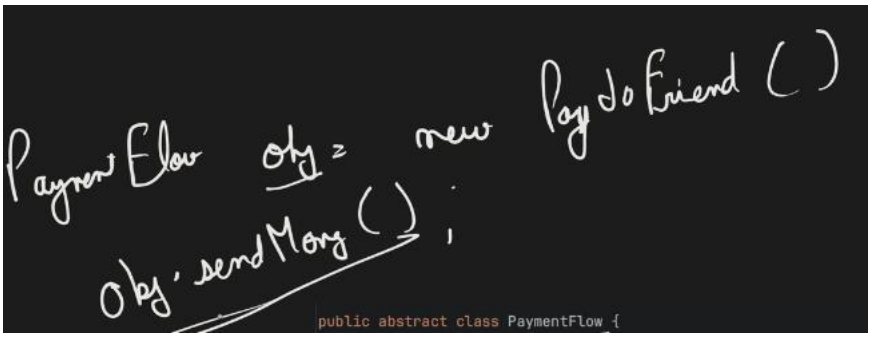
```java
public abstract class PaymentFlow {

    public abstract void validateRequest();
    public abstract void calculateFees();
    public abstract void debitAmount();
    public abstract void creditAmount();

    //this is Template method: which defines the order of steps to execute the task.
    public final void sendMoney(){
        //step1
        validateRequest();

        //step2
        debitAmount();

        //step3
        calculateFees();

        //step4
        creditAmount();
    }
}
```

```java
public class PayToFriend extends PaymentFlow{

    @Override
    public void validateRequest() {
        //specific validation for PayToFriend flow
        System.out.println("Validate logic of PayToFriend");
    }

    @Override
    public void debitAmount() {
        //debit the amount
        System.out.println("Debit the Amount logic of PayToFriend");
    }

    @Override
    public void calculateFees() {
        //specific Fee computation logic for PayToFriend flow
        System.out.println("0% fees charged");
    }

    @Override
    public void creditAmount() {
        // credit the amount logic
        System.out.println("Credit the full amount");
    }
```

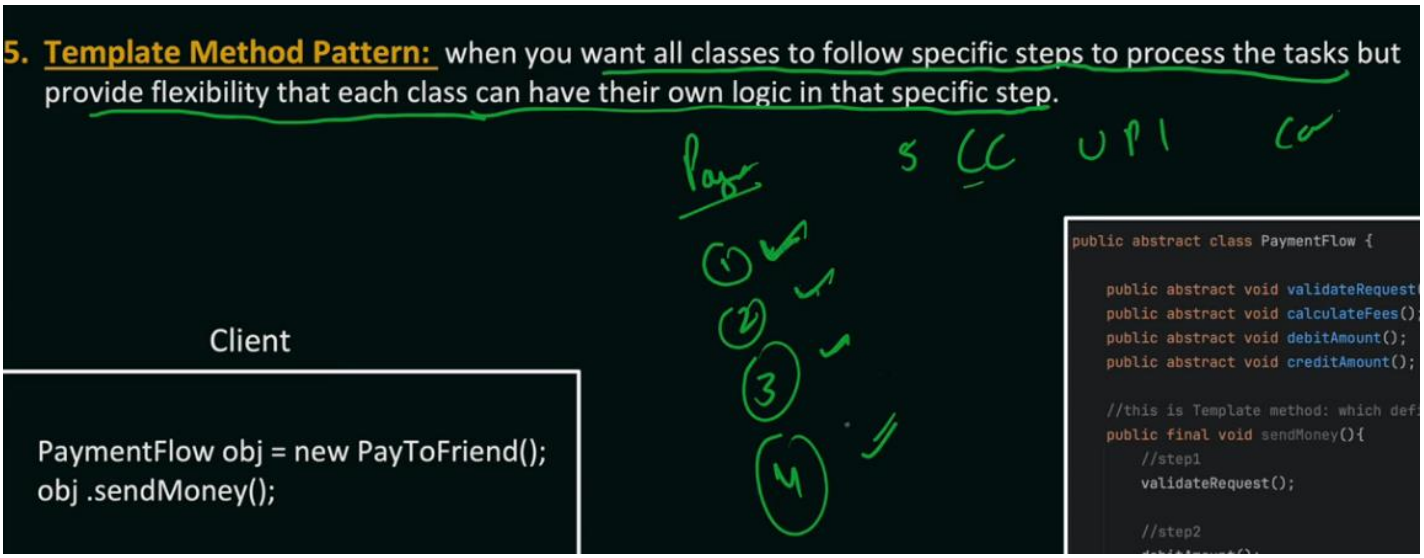
```java
public class PayToMerchantFlow extends Payment

    @Override
    public void validateRequest() {
        //specific validation for PayToFriend
        System.out.println("Validate logic of
    }

    @Override
    public void debitAmount() {
        //debit the amount
        System.out.println("Debit the Amount l
    }

    @Override
    public void calculateFees() {
        //specific Fee computation logic for Pay
        System.out.println("2% fees charged");
    }

    @Override
    public void creditAmount() {
        // credit the amount logic
        System.out.println("Credit the remaining
    }
```

template method - is providing send money task and I have created final that no child class can override it.
So the child classes have to follow this order.
They can't provide at the rate override send money method.

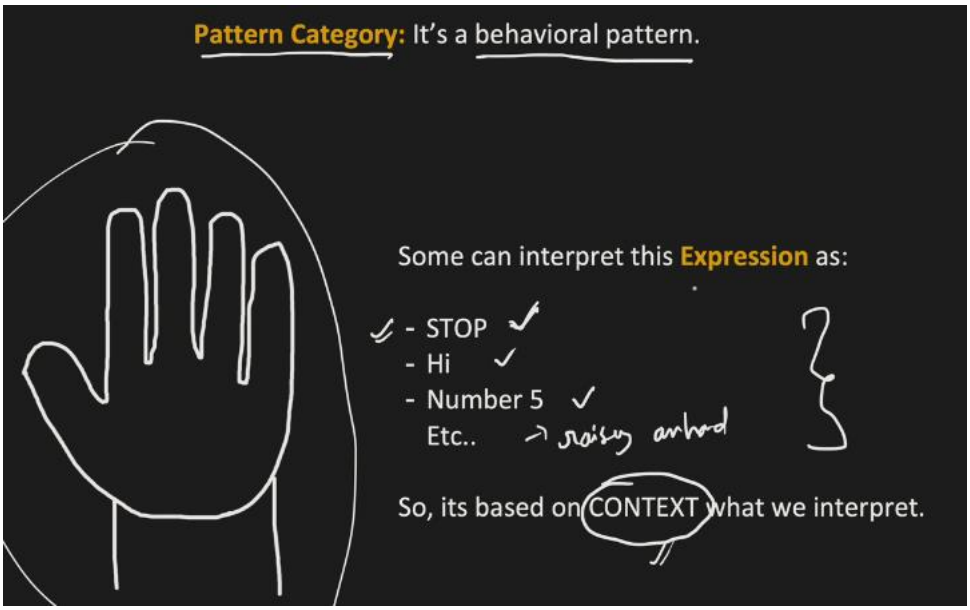


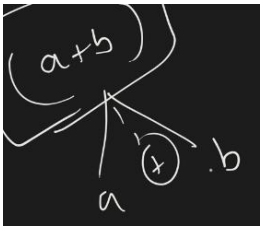


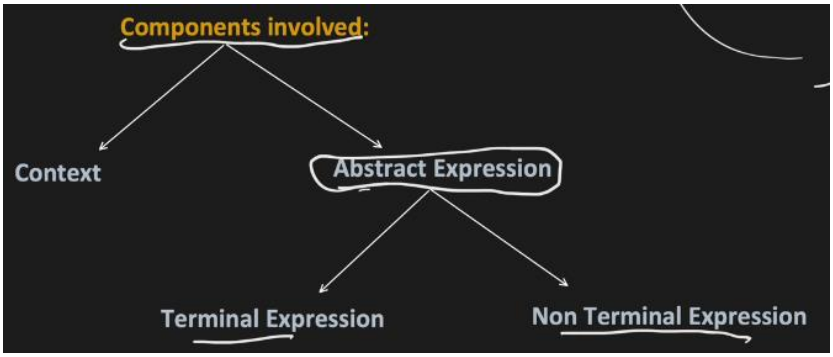
https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/behavioralpatterns/templatemethod?ref_type=heads

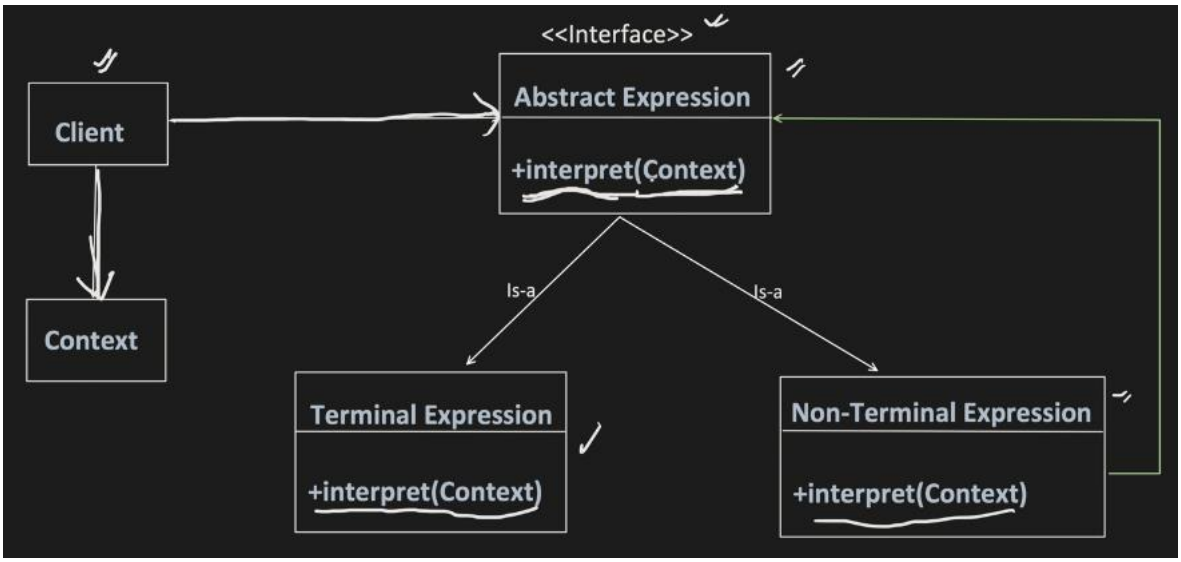


36. Interpreter Pattern (Behavioral Design Pattern)









Example 1 :

EXAMPLE 1:

$a = 10$

$a = 11$

$(a * b)$??

Expression

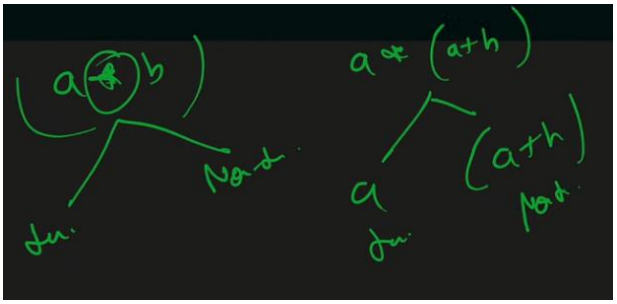
```java
public class Client {

    public static void main(String args[]) {

        //initialize the context
        Context context = new Context();
        context.put( strVariable: "a",  intValue: 2);
        context.put( strVariable: "b",  intValue: 4);

        //a*b
        AbstractExpression expression1 = new MultiplyNonTerminalExpression(
                new NumberTerminalExpression( stringVal: "a"), new NumberTerminalExpression( stringVal: "b"));

        System.out.println(expression1.interpret(context));
    }
}
```

```java
public i

    int :
}
```

```java
public static void main(String args[]) {

    //initialize the context
    Context context = new Context();
    context.put( strVariable: "a",  intValue: 2);
    context.put( strVariable: "b",  intValue: 4);

    //a*b
    AbstractExpression expression1 = new MultiplyNonTerminalExpression(
            new NumberTerminalExpression( stringVal: "a"), new NumberTerminalExpression( stringVal: "b

    System.out.println(expression1.interpret(context));
}
}
```

a	2
b	4

```java
public class Context {

    Map<String, Integer> contextMap = new HashMap<>();

    public void put(String strVariable, int intValue) {
        contextMap.put(strVariable, intValue);
    }

    public int get(String strVariable) {
        return contextMap.get(strVariable);
    }
}
```




```java
public interface AbstractExpression {

    int interpret(Context context);
}
```

$(a * b)$

is-a

```java
public class NumberTerminalExpression implements AbstractExpression{

    String stringValue;

    NumberTerminalExpression(String stringVal) {
        this.stringValue = stringVal;
    }

    @Override
    public int interpret(Context context) {
        return context.get(stringValue);
    }
```

```java
public class MultiplyNonTerminalExpression implements AbstractExpression{

    AbstractExpression leftExpression;
    AbstractExpression rightExpression;

    public MultiplyNonTerminalExpression(AbstractExpression leftExpression, Abstract
        this.leftExpression = leftExpression;
        this.rightExpression = rightExpression;
    }

    @Override
    public int interpret(Context context) {
        return leftExpression.interpret(context) * rightExpression.interpret(context
    }
```

udemy

Example : 2 (same context)

```java
public interface AbstractExpression {

    int interpret(Context context);
}
```

```java
public class NumberTerminalExpression implements AbstractExpression{

    String stringValue;

    NumberTerminalExpression(String stringVal) {
        this.stringValue = stringVal;
    }

    @Override
    public int interpret(Context context) {
        return context.get(stringValue);
    }
}
```

```java
public class MultiplyNonTerminalExpression implements AbstractExpression{

    AbstractExpression leftExpression;
    AbstractExpression rightExpression;

    public MultiplyNonTerminalExpression(AbstractExpression leftExpression, AbstractExpression rightExpression) {
        this.leftExpression = leftExpression;
        this.rightExpression = rightExpression;
    }

    @Override
    public int interpret(Context context) {
        return leftExpression.interpret(context) * rightExpression.interpret(context);
    }
}
```

```java
public class SumNonTerminalExpression implements AbstractExpression{

    AbstractExpression leftExpression;
    AbstractExpression rightExpression;

    public SumNonTerminalExpression(AbstractExpression leftExpression, Abstract
        this.leftExpression = leftExpression;
        this.rightExpression = rightExpression;
    }

    @Override
    public int interpret(Context context) {
        return leftExpression.interpret(context) + rightExpression.interpret(cor
    }
}
```

```java
public static void main(String args[]) {

    //initialize the context
    Context context = new Context();
    context.put( strVariable: "a",  intValue: 2);
    context.put( strVariable: "b",  intValue: 4);
    context.put( strVariable: "c",  intValue: 8);
    context.put( strVariable: "d",  intValue: 16);

    // ((a *b) + (c*d))
    AbstractExpression expression2 = new SumNonTerminalExpression(
            new MultiplyNonTerminalExpression(new NumberTerminalExpression( stringVal: "a"), new NumberTerminalExpression( stringVal: "b")),
                new MultiplyNonTerminalExpression(new NumberTerminalExpression( stringVal: "c"), new NumberTerminalExpression( stringVal: "d")));
    System.out.println(expression2.interpret(context));
}
```

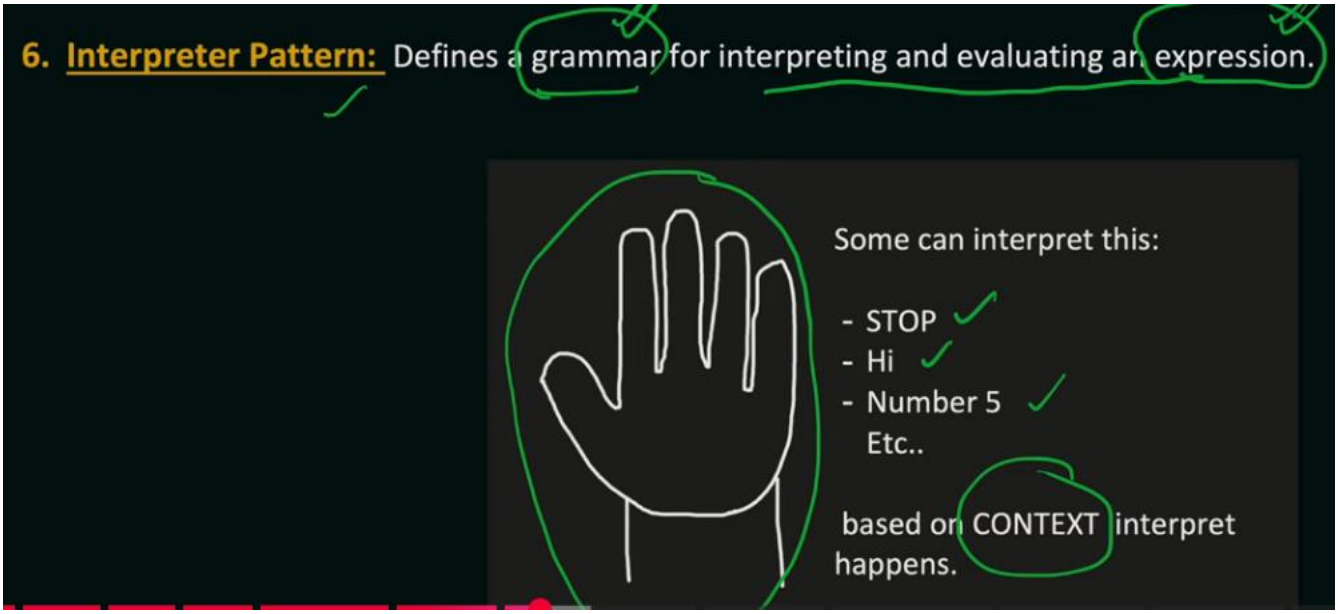
Example 3: (same context)

Further optimization by putting all the non-terminal expressions grouped in one class

```java
public class BinaryNonTerminalExpression implements AbstractExpression{

    AbstractExpression leftExpression;
    AbstractExpression rightExpression;
    char operator;

    public BinaryNonTerminalExpression(AbstractExpression leftExpression, AbstractExpression rightExpressi
        this.leftExpression = leftExpression;
        this.rightExpression = rightExpression;
        this.operator = operator;
    }

    @Override
    public int interpret(Context context) {
        switch (operator) {
            case '+':
                return leftExpression.interpret(context) + rightExpression.interpret(context);
            case '*':
                return leftExpression.interpret(context) * rightExpression.interpret(context);
            default:
                return 0;
        }
    }
}
```

```java
public class Client {

    public static void main(String args[]) {

        //initialize the context
        Context context = new Context();
        context.put( strVariable: "a",  intValue: 2);
        context.put( strVariable: "b",  intValue: 4);
        context.put( strVariable: "c",  intValue: 8);
        context.put( strVariable: "d",  intValue: 16);

        // ((a *b) + (c*d))
        AbstractExpression expression2 = new BinaryNonTerminalExpression(
                new BinaryNonTerminalExpression(
                        new NumberTerminalExpression( stringVal: "a"), new NumberTerminalExpression( stringVal: "b"),  operator: '*'),
                        new BinaryNonTerminalExpression(
                                new NumberTerminalExpression( stringVal: "c"), new NumberTerminalExpression( stringVal: "d"),  operator: '*'),
                operator: '+');
        System.out.println(expression2.interpret(context));
    }
}
```

((a, b, *) (c, d, *) +)



https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/behavioralpatterns/interpreter?ref_type=heads